# M12: Your First Script

> "If you can type it, you can automate it."

**Mission:** You've been SSH-ing to servers and typing commands manually. Time to write your first script and run it once instead of 100 times.

# Quick Check

**Question:** Who has written a bash script before?

- Show of hands
- Don't worry if you haven't — by the end of today, you'll be automating everything

**Follow-up:** What's the most repetitive task you do on Linux?

> That's exactly what scripts solve.

# After This Module, You Will Be Able To:

- Write and execute your first bash script
- Use variables to store and reuse data
- Understand quoting rules (single vs double quotes)
- Capture command output with `$()`
- Use environment variables like `$USER` and `$HOME`
- Export variables to child processes
- Source scripts vs executing them

**This is how automation starts.**

# Why Automation Matters in Cloud

| Scenario | Manual Approach | Automated Approach |
|---|---|---|
| Configure 1 server | Type commands (10 min) | Write script (10 min) |
| Configure 10 servers | Type commands × 10 (100 min) | Run script × 10 (2 min) |
| Configure 100 servers | **Impossible** | Run script × 100 (5 min) |
| Server crashes at 3 AM | Wake up, fix it | Script already fixed it |

**The Three Pillars:**

### Scale

One script, 1,000 servers

### Consistency

No "I forgot a step on server 47"

### Repeatability

Version control your infrastructure

# What Is a Bash Script?

A bash script is a **text file** containing the same commands you type in the terminal.

```
# Instead of typing these one by one:
sudo apt update
sudo apt install -y nginx
sudo systemctl enable --now nginx

# Put them in a file called setup.sh:
#!/bin/bash
sudo apt update
sudo apt install -y nginx
sudo systemctl enable --now nginx
```

**Run once:**

```
./setup.sh
```

**Why scripts beat typing:**

- Repeatable (run 100 times without retyping)
- Shareable (give to your team)
- Version controlled (git tracks changes)
- Schedulable (cron runs them automatically)

# The Shebang: #!/bin/bash

The **first line** of every script tells Linux which interpreter to use:

```
#!/bin/bash
echo "Hello from Bash!"
```

| Shebang | Interpreter | When to Use |
|---|---|---|
| `#!/bin/bash` | Bash | Most scripts (standard) |
| `#!/usr/bin/env bash` | Bash (portable) | Multi-distro scripts |
| `#!/bin/sh` | POSIX shell | Minimal scripts |
| `#!/usr/bin/env python3` | Python 3 | Python scripts |

**Even system commands are scripts:**

```
anas@tuwaiq:~$ head -1 /usr/bin/apt
#!/usr/bin/python3
```

> **Pro tip:** Use `#!/usr/bin/env bash` for portability across different systems.

# Creating and Running Scripts

**Step 1: Create the file**

```
anas@tuwaiq:~$ nano hello.sh
```

**Step 2: Write the script**

```
#!/bin/bash
echo "Hello from my first script!"
echo "Today is $(date)"
echo "I am user: $USER on host: $(hostname)"
```

**Step 3: Make it executable**

```
anas@tuwaiq:~$ chmod +x hello.sh
```

**Step 4: Run it**

```
anas@tuwaiq:~$ ./hello.sh
Hello from my first script!
Today is Mon Feb 10 14:30:00 UTC 2026
I am user: anas on host: tuwaiq
```

# Three Ways to Run Scripts

```
# Method 1: Relative path (most common)
anas@tuwaiq:~$ ./hello.sh

# Method 2: Absolute path
anas@tuwaiq:~$ /home/anas/scripts/hello.sh

# Method 3: Using bash (no chmod needed)
anas@tuwaiq:~$ bash hello.sh

# Method 4: source (runs in current shell)
anas@tuwaiq:~$ source hello.sh
# or
anas@tuwaiq:~$ . hello.sh
```

**Why `./` before the filename?**

- Linux only looks in `$PATH` directories for commands
- Your current directory is NOT in `$PATH` (for security)
  `./` means "look in the current directory"

> **Key difference:** `./script.sh` runs in a NEW shell. `source script.sh` runs in your CURRENT shell (variables persist).

# Make Your Scripts Fail Fast

**Problem:** By default, bash continues running even after errors!

```
#!/bin/bash
cd /nonexistent          # This fails...
rm -rf *                  # ...but this still runs!
```

**Solution: Use `set -e` at the start of EVERY script**

```
#!/bin/bash
set -e   # Exit immediately if any command fails

cd /nonexistent          # Script STOPS here
rm -rf *                  # This never runs
```

| Without `set -e` | With `set -e` |
|---|---|
| Keeps running after errors | Stops on first error |
| Silent failures | Loud failures (you notice) |
| Dangerous in production | Safe by default |

> **Best practice:** Add `set -e` right after the shebang in every script. We'll learn more error handling options later (M14).

# Variables: Storing Data

```
# Setting variables (NO spaces around =)
NAME="Anas"
VERSION=22.04
SERVER_COUNT=10

# Using variables ($ prefix)
echo $NAME                      # Anas
echo $VERSION                   # 22.04
echo "User: $NAME"             # User: Anas

# Curly braces for clarity
echo "File: ${NAME}_config.txt"      # Anas_config.txt
echo "Ubuntu ${VERSION}LTS"          # Ubuntu 22.04LTS
```

**Common mistake:**

```
NAME = "Anas"     #
```

**Variable naming rules:**

- Letters, numbers, underscores only
- Cannot start with a number
- Convention: `UPPER_CASE` for constants, `lower_case` for local

# Quoting: Single vs Double vs None

```
NAME="Anas"
FILE="my document.txt"

# Double quotes — variables EXPAND, spaces preserved
echo "Hello, $NAME"              # Hello, Anas
echo "File: $FILE"               # File: my document.txt

# Single quotes — NOTHING expands, everything literal
echo 'Hello, $NAME'              # Hello, $NAME
echo 'Price: $5.00'              # Price: $5.00

# No quotes — word splitting happens!
echo $FILE                       # my document.txt (two arguments!)
```

| Quote Type | Variables Expand? | Spaces Safe? | Use When |
|---|---|---|---|
| `"double"` | ✅ Yes | ✅ Yes | Most of the time |
| `'single'` | ❌ No | ✅ Yes | Literal strings, regex |
| None | ✅ Yes | ❌ **NO!** | Almost never |

> **Rule of thumb:** When in doubt, use double quotes. **Always quote variables:** `"$VAR"` not `$VAR`.

# Security: Always Quote Variables

**Unquoted variables are a security risk!**

```
# ── DANGEROUS ──────────────────────────────
DIR=""                          # empty — could happen from a failed command
rm -rf $DIR/*                   # expands to: rm -rf /*  💀

DIR="my backups"                # spaces — could happen from user input
rm -rf $DIR/*                   # expands to: rm -rf my backups/*  💀
                                # two arguments: "my" and "backups/*"

# ── SAFE (quotes) ──────────────────────────
DIR="my backups"
rm -rf "${DIR}"/*               # expands to: rm -rf "my backups"/*  ✅
                                # one argument, spaces preserved
# ── SAFEST (:?) ────────────────────────────
DIR=""
rm -rf "${DIR:?Error: DIR not set or empty}"/*       # bash: DIR: Error: DIR not set → exits immediately, rm never runs  ✅
```

**Real-world example:**

```
# Vulnerable to injection
FILENAME="$1"
cat $FILENAME              # User can inject: "file; rm -rf /"

# Safe
cat "$FILENAME"            # Treated as single filename
```

> **Aha! Moment:** Quoting isn't just syntax — it's security. One missing quote can delete your entire filesystem. Always quote `"$VARIABLES"`.

# Command Substitution: Capture Output

**Syntax:** `$(command)` captures the output of a command into a variable

```
# Get current date
TODAY=$(date +%Y-%m-%d)
echo "Backup file: backup-${TODAY}.tar.gz"
# Output: Backup file: backup-2026-02-10.tar.gz

# Get hostname
SERVER=$(hostname)
echo "Running on: $SERVER"

# Count files
FILE_COUNT=$(ls -1 /var/log/*.log | wc -l)
echo "Found $FILE_COUNT log files"

# Get system info
UPTIME=$(uptime -p)
DISK_USAGE=$(df -h / | awk 'NR==2 {print $5}')
echo "System up: $UPTIME, Disk: $DISK_USAGE"
```

**Old syntax (avoid):** `` `command` `` **Modern syntax (use):** `$(command)`

> **Pro tip:** You can nest command substitutions: `echo "Lines in apt: $(wc -l < $(which apt))"`

# Special Variables

**Positional arguments:**

| Variable | Meaning | Example |
|---|---|---|
| `$0` | Script name | `./backup.sh` |
| `$1`, `$2`, ... | Arguments | First arg, second arg |
| `$#` | Number of arguments | `3` |
| `$@` | All arguments (separate) | `"arg1" "arg2"` |

**Environment variables (always available):**

| Variable | Meaning | Example |
|---|---|---|
| `$USER` | Current username | `anas` |
| `$HOME` | Home directory | `/home/anas` |
| `$PWD` | Current directory | `/home/anas/scripts` |
| `$PATH` | Command search paths | `/usr/bin:/bin` |
| `$HOSTNAME` | Machine name | `tuwaiq` |

**Example:**

```
echo "Running $0 as $USER in $PWD"
# Output: Running ./backup.sh as anas in /home/anas/scripts
```

# Environment Variables: export

**Problem:** Variables are local to the current shell by default.

```
# Local variable (only in this script)
NAME="Anas"
./child-script.sh    # child-script.sh can't see $NAME

# Environment variable (available to child processes)
export NAME="Anas"
./child-script.sh    # child-script.sh CAN see $NAME
```

**When to export:**

```
#!/bin/bash
# deploy.sh — Pass config to child scripts

export ENVIRONMENT="production"
export APP_NAME="myapp"
export DB_HOST="db.example.com"

# These child scripts can now use $ENVIRONMENT, $APP_NAME, $DB_HOST
./backup-database.sh
./deploy-app.sh
./health-check.sh
```

**Check environment:**

```
env                  # Show all environment variables
printenv HOME        # Show specific variable
```

> Variables vs Environment: Think "local scope" vs "global scope" in programming.

# Sourcing Scripts: Sharing Variables

**Two ways to run scripts:**

```
# Execute (runs in NEW shell)
./script.sh
# Variables and changes stay in child shell, don't affect current shell

# Source (runs in CURRENT shell)
source script.sh
# or
. script.sh
# Variables and changes affect current shell
```

**Example:**

```
# config.sh
#!/bin/bash
DB_HOST="localhost"
DB_USER="admin"
API_KEY="secret123"

# main.sh
#!/bin/bash
source ./config.sh    # Load variables from config.sh
echo "Connecting to $DB_HOST as $DB_USER"
```

**Common pattern:**

```
# Source function libraries
source /usr/local/lib/deploy-functions.sh
log "Deployment started"
check_service "nginx"
```

# Practical Example: System Info Script

```
#!/bin/bash
set -e

# Get system information
HOSTNAME=$(hostname)
UPTIME=$(uptime -p)
MEMORY_USAGE=$(free -h | awk 'NR==2 {print $3 "/" $2}')
DISK_USAGE=$(df -h / | awk 'NR==2 {print $5}')
CPU_COUNT=$(nproc)
CURRENT_USER=$USER

# Display report
echo "======================================"
echo "  System Information Report"
echo "======================================"
echo "Hostname:      $HOSTNAME"
echo "Uptime:        $UPTIME"
echo "Current User:  $CURRENT_USER"
echo "CPU Cores:     $CPU_COUNT"
echo "Memory Usage:  $MEMORY_USAGE"
echo "Disk Usage:    $DISK_USAGE"
echo "======================================"
echo "Report generated: $(date)"
```

**Try it:**

```
chmod +x system-info.sh
./system-info.sh
```

# Aha! Moment 💡

> **"If you can type it in the terminal, you can automate it in a script."**

Every command you've learned in Day 1 and Day 2:

- `ls`, `cd`, `cat`, `grep`
- `apt install`, `systemctl restart`
- `chmod`, `useradd`, `sudo`

**ALL of these can go in a script.**

```
#!/bin/bash
# One script to rule them all

sudo apt update
sudo apt install -y nginx
sudo systemctl enable --now nginx
sudo ufw allow 'Nginx Full'
echo "Web server is ready!"
```

**You're not typing commands anymore. You're writing infrastructure.**

# Mission Complete! ✅

**What you learned:**

- ✅ Create and execute bash scripts (`chmod +x`, `./script.sh`)
- ✅ Use variables to store data (`NAME="value"`)
- ✅ Quote variables correctly (`"$VAR"` always!)
- ✅ Capture command output (`$(command)`)
- ✅ Use environment variables (`$USER`, `$HOME`, `export`)
- ✅ Source vs execute scripts
- ✅ `set -e` to fail fast

**Next up: M13 — Script Logic: Decisions & Loops**

Your script can run commands. But what if you need to check if nginx is installed first? What if a file doesn't exist? What if a service is down?

**Time to add logic to your scripts.**

# Questions?

**Common questions:**

- Why `./` before script names?
- When to use `export`?
- Single vs double quotes — which should I use?
- How do I debug scripts?

**Resources:**

- Bash manual: `man bash`
- Test your scripts: https://www.shellcheck.net
- Quote tester: https://explainshell.com